

Módulo de Captura de Tráfego e Geração de Fluxos (VM4)

Gustavo Assis Ferreira
Bacharelado em Engenharia Elétrica
Instituto Federal do Espírito Santo
Guarapari, ES

Resumo—Este relatório técnico individual apresenta o desenvolvimento do módulo distribuído de Captura de Tráfego (VM4), integrante de uma infraestrutura de rede distribuída em ambiente OpenStack. O projeto englobou a criação da aplicação denominada *Flow Generator*, uma aplicação em Python projetada para realizar a escuta passiva de pacotes, decodificar múltiplas sub-redes virtuais (VLANs), agrupar as comunicações em fluxos bidirecionais e exportar as estatísticas computadas para uma API REST centralizada. O documento detalha a arquitetura da solução, as configurações de sistema operacional aplicadas para contornar restrições de *Port Security*, as justificativas pela escolha e aplicação de cada arquitetura envolvendo concorrência de *threads*, as etapas de validação, containerização via Docker e resultados obtidos.

Index Terms—Monitoramento de Redes, OpenStack, Geração de Fluxos, Scapy, Docker, Port Mirroring.

I. INTRODUÇÃO

A visibilidade de tráfego em infraestruturas de Redes Definidas por Software (SDN) e ambientes em nuvem, como o OpenStack, apresenta desafios singulares devido à abstração do roteamento físico. Para garantir a monitoria e a segurança dessas topologias virtuais, faz-se necessário o emprego de aplicações de captura estrategicamente posicionadas, capazes de receber e interpretar o tráfego espelhado (*Port Mirroring*) sem interferir na comunicação original.

Este relatório detalha a implementação da Máquina Virtual 4 (VM4), que atua como o módulo distribuído de captura de tráfego do trabalho proposto da disciplina de Laboratório de Redes. A VM4 foi encarregada de processar passivamente os pacotes provenientes de múltiplas redes virtuais (VLANs) da corporação simulada e transformar esses dados brutos recebidos do espelhamento de dois switches virtuais em métricas de fluxo estruturadas para o consumo do servidor central (VM1) e de outros módulos distribuídos propostos no trabalho.

II. OBJETIVOS

O objetivo central deste trabalho consistiu no desenvolvimento e na implementação do módulo distribuído de Captura de Tráfego, hospedado na Máquina Virtual 4 (VM4). Especificamente, o projeto visou a criação da aplicação com nome escolhido pelo aluno 4, denominado *Flow Generator*, que é responsável por:

- Realizar a escuta passiva do tráfego de rede espelhado a partir dos *switches* virtuais.

- Decodificar pacotes encapsulados, identificando automaticamente a rede virtual (VLAN) correspondente através da interface de escuta.
- Agrupar os pacotes em fluxos de comunicação consolidados.
- Computar métricas de tráfego, incluindo contabilização de pacotes e bytes, duração do fluxo, taxa de transmissão (bytes/s), protocolo e serviço associado.
- Exportar os resultados processados para um arquivo JSON local e transmitir ativamente os fluxos estruturados, resumo e estatísticas para uma API REST hospedada na VM1.
- Garantir o encapsulamento e a portabilidade do sistema através da containerização via Docker.

III. METODOLOGIA

A metodologia adotada para o desenvolvimento deste trabalho foi organizada em etapas que refletem a sequência lógica de implementação da solução proposta. Dessa forma, esta seção encontra-se subdividida em tópicos que descrevem desde a preparação do ambiente experimental, passando pela escolha das tecnologias empregadas, desenvolvimento da aplicação *Flow Generator*, integração com a API REST, até os procedimentos de validação e testes realizados para verificar o correto funcionamento do sistema.

A. Preparação do Ambiente

O desenvolvimento deste trabalho foi realizado de forma gradual, passando por diferentes etapas de implementação, testes e ajustes até chegar à versão final da aplicação. A primeira etapa consistiu na criação da máquina virtual principal (VM4), responsável pela execução do *Flow Generator*, e de uma máquina virtual auxiliar de testes, denominada *VMTESTE (Aluno4)*, utilizada para gerar tráfego de rede e validar o funcionamento da aplicação durante o desenvolvimento.

As duas máquinas foram criadas no ambiente OpenStack utilizando o sistema operacional Ubuntu Server 24.04 LTS. Para ambas foi utilizado o mesmo perfil de hardware adotado nos laboratórios da disciplina, composto por 1 vCPU, 1 GB de memória RAM e 16 GB de armazenamento em disco. As Figuras 1, 3 e 2 apresentam a criação da instância, o *flavor* escolhido e a imagem do sistema operacional utilizada. Já a Figura 4 mostra as redes e interfaces conectadas a cada uma das máquinas virtuais utilizadas durante o desenvolvimento.

Inicialmente, as máquinas foram conectadas apenas à rede *LabRedes*, que possui acesso à Internet. Essa etapa foi necessária para instalar o sistema operacional, atualizar os repositórios e baixar todas as ferramentas utilizadas durante o desenvolvimento, como Python, Docker, Scapy e outras bibliotecas necessárias para a aplicação.

A rede *LabRedes* utiliza a faixa de endereços 192.168.10.0/24, enquanto os computadores do laboratório pertencem à rede 10.10.0.0/24. Para permitir o acesso remoto à VM4 a partir dos computadores do laboratório, foi associado à máquina um endereço IP flutuante (*Floating IP*) 10.10.1.22. Dessa forma, foi possível acessar a máquina via SSH durante todo o desenvolvimento do projeto.

O acesso por SSH facilitou bastante o desenvolvimento quando comparado ao console disponibilizado pelo OpenStack, pois permitiu utilizar o terminal local, copiar arquivos entre o computador e a máquina virtual, executar comandos com maior facilidade e realizar a depuração da aplicação de forma muito mais prática.

Após a instalação das ferramentas e a realização dos testes iniciais de conectividade, o ambiente estava preparado para iniciar o desenvolvimento da aplicação *Flow Generator* e sua posterior integração com a infraestrutura de rede desenvolvida pelos demais integrantes do projeto.

Figura 1. Seleção da imagem Ubuntu 24.04 na configuração da instância.

B. Escolha da Ferramenta de Captura

Após a preparação do ambiente experimental e a configuração inicial das máquinas virtuais, iniciou-se a etapa de definição da tecnologia responsável pela captura e processamento dos pacotes de rede. Essa escolha representou uma das decisões mais importantes do projeto, uma vez que todo o funcionamento do módulo de geração de fluxos dependeria diretamente da capacidade da ferramenta em capturar, interpretar e disponibilizar as informações presentes nos pacotes de maneira eficiente.

Inicialmente foram avaliadas diferentes alternativas amplamente utilizadas em ambientes Linux para monitoramento de redes. Entre elas destacaram-se o comando de terminal

Figura 2. Especificações do flavor selecionado para a máquina virtual

Figura 3. Flavor selecionado para a máquina virtual.

VM_TESTE (Aluno4)	Ubuntu 24.04	192.168.10.166, 10.10.3.145	minor.pico.large	ubuntu
VM4	Ubuntu 24.04	VLAN40_CAP.TRAF2 10.0.90.231 VLAN40_CAP.TRAFE 10.0.190.169 VLAN40_CAP.TRAF1 10.0.140.186 labredes1 192.168.10.193, 10.10.3.225	minor.pico.large	ubuntu

Figura 4. VM4 e VMTESTE

tcpdump, o analisador de protocolos TShark, sua interface para Python denominada PyShark, e a biblioteca Scapy.

O tcpdump mostrou-se extremamente eficiente para validar rapidamente a presença de tráfego nas interfaces de rede e confirmar o funcionamento do espelhamento configurado pelos switches virtuais. Sua execução diretamente no terminal permitiu verificar a chegada dos pacotes nas interfaces monitoradas, tornando-se uma ferramenta importante durante toda a fase de depuração da infraestrutura. Entretanto, por tratar-se de um programa voltado exclusivamente à captura e exibição de pacotes em linha de comando, sua utilização como componente principal da aplicação exigiria mecanismos adicionais para interpretar sua saída textual, aumentando significativamente a complexidade do desenvolvimento.

Em seguida foram realizados testes utilizando o TShark, versão em linha de comando do Wireshark, e posteriormente sua interface para Python, o PyShark. Essas ferramentas oferecem uma poderosa estrutura de dissecação de protocolos, suportando centenas de protocolos de rede e fornecendo diversas informações já decodificadas. Apesar dessas vantagens, observou-se que ambas dependem da instalação do próprio TShark como processo externo ao interpretador Python, fa-

zendo com que a aplicação precisasse invocar processos auxiliares continuamente para capturar o tráfego. Essa arquitetura introduz maior consumo de memória, dependências adicionais e uma camada extra de comunicação entre o programa principal e o processo responsável pela captura.

Após essa etapa comparativa, optou-se pela utilização da biblioteca Scapy como tecnologia principal do projeto. O Scapy é uma biblioteca escrita inteiramente em Python destinada à manipulação, criação, envio, recepção e dissecação de pacotes de rede. Diferentemente das demais soluções avaliadas, sua utilização ocorre de maneira totalmente integrada ao código da aplicação, dispensando a execução de processos externos durante a captura.

Outro fator determinante para essa escolha foi a flexibilidade oferecida pelo Scapy. A biblioteca permite acessar diretamente todos os campos presentes nos cabeçalhos dos protocolos IP, TCP, UDP e ICMP através de objetos Python, possibilitando que a própria aplicação realize a interpretação das informações necessárias para construção dos fluxos de comunicação. Com isso, será apresentado no tópico seguinte o passo a passo feito para o desenvolvimento do flow generator.

C. Desenvolvimento do Flow Generator

Definida a tecnologia de captura, iniciou-se o desenvolvimento da aplicação denominada Flow Generator, componente responsável por transformar o tráfego bruto recebido das interfaces de rede em informações estruturadas capazes de representar os fluxos de comunicação existentes na infraestrutura monitorada.

A implementação foi realizada integralmente na linguagem Python, adotando uma arquitetura modular que facilitasse futuras expansões da aplicação. Desde o início do desenvolvimento buscou-se separar as responsabilidades do sistema em pequenas funções, cada uma encarregada de executar uma tarefa específica, como captura dos pacotes, identificação dos protocolos, agrupamento dos fluxos, exportação dos dados e comunicação com a API REST.

Inicialmente foi realizada a atualização dos repositórios do sistema operacional, conforme apresentado no Comando III-C.

Atualização dos repositórios do Ubuntu

```
apt update
```

Em seguida, foi feita a instalação do gerenciador de pacotes Python e da biblioteca Scapy.

Instalação do python3

```
apt install python3 python3-pip
```

Instalação do Scapy

```
apt install Scapy
```

Após a instalação da biblioteca *Scapy*, iniciou-se o desenvolvimento da aplicação denominada *Flow Generator*. A primeira etapa consistiu na importação das bibliotecas necessárias para

o funcionamento do sistema. Cada biblioteca foi selecionada para desempenhar uma função específica dentro da arquitetura da aplicação, conforme apresentado na Listagem 1.

Listing 1. Importação das bibliotecas utilizadas pelo Flow Generator.

```
import requests
from scapy.all import sniff, IP, TCP, UDP, ICMP
from datetime import datetime
import threading
import time
import json
import os
```

Cada uma das bibliotecas apresentadas na Listagem 1 possui uma finalidade específica dentro da aplicação:

- **Requests:** utilizada para realizar requisições HTTP do tipo POST, responsáveis pelo envio dos fluxos processados para a API REST hospedada na VM1.
- **Scapy:** biblioteca responsável pela captura e dissecação dos pacotes de rede. Além de permitir a escuta das interfaces monitoradas, fornece acesso direto aos protocolos IP, TCP, UDP e ICMP, possibilitando a extração dos endereços IP, portas e demais informações utilizadas na construção dos fluxos.
- **Datetime:** utilizada para registrar o instante de chegada de cada pacote capturado, permitindo posteriormente calcular a duração de cada fluxo de comunicação.
- **Threading:** empregada para executar tarefas concorrentes. Enquanto uma thread permanece dedicada exclusivamente à captura contínua dos pacotes, outra thread executa periodicamente a impressão da tabela de fluxos, exportação dos dados e envio das informações para a API da VM1.
- **Time:** utilizada para controlar intervalos de execução da thread responsável pelo monitoramento dos fluxos, evitando consumo excessivo de processamento.
- **JSON:** utilizada para converter os dados estruturados em formato JSON, permitindo tanto a exportação dos fluxos para um arquivo local quanto o envio dessas informações para a API.
- **OS:** biblioteca utilizada para operações auxiliares relacionadas ao sistema operacional, permitindo maior flexibilidade na manipulação de arquivos e diretórios quando necessário.

Assim sendo, foram definidas algumas configurações globais que seriam utilizadas durante toda a execução da aplicação, como o endereço IP da máquina responsável pelo servidor central (VM1), o endpoint da API REST para envio dos fluxos e o tempo máximo de inatividade permitido para um fluxo permanecer ativo na memória.

Por fim, também foram declaradas as estruturas responsáveis por armazenar o estado da aplicação durante a execução, incluindo o conjunto de fluxos ativos, o controle dos fluxos já enviados para a API e os contadores globais de pacotes e bytes capturados.

Listing 2. Definição das configurações globais da aplicação.

```
VM1_IP = "10.0.20.10"
VM1_API = f"http://{VM1_IP}/api/v1/flows/batch"
FLOW_TIMEOUT = 60

flows = {}
sent_flows = set()
flows_changed = False
total_packets = 0
total_bytes = 0
```

A variável `VM1_IP` armazena o endereço IP do servidor responsável por receber os fluxos processados, enquanto `VM1_API` define o endpoint utilizado para envio dos dados em lote (*batch*). O parâmetro `FLOW_TIMEOUT` determina o tempo máximo de inatividade permitido para um fluxo antes de ser removido da memória.

O dicionário `flows` representa a principal estrutura de armazenamento da aplicação, sendo responsável por manter todas as conexões ativas observadas durante a captura. O conjunto `sent_flows` registra quais fluxos já foram transmitidos para a API, evitando envios duplicados.

Por fim, as variáveis `total_packets` e `total_bytes` mantêm estatísticas globais da captura, enquanto a variável booleana `flows_changed` sinaliza quando novos pacotes foram recebidos, permitindo que apenas alterações relevantes sejam processadas pela thread responsável pela exportação dos dados.

1) *Comunicação com a API REST*: Responsável pela integração externa, esta função recebe os lotes de fluxos processados e utiliza requisições HTTP POST para enviá-los à VM1 em formato JSON. Ela foi construída com tratamento de erros e validação dos códigos de resposta HTTP (como 200 e 201), garantindo que falhas momentâneas na rede não paralise o programa principal.

Listing 3. Núcleo da requisição POST para a API.

```
def send_flow(batch):
    try:
        response = requests.post(
            VM1_API,
            json=batch,
            timeout=5
        )
        if response.status_code in [200, 201]:
            print(f"[VM1 OK] {len(batch)} fluxos enviados.")
        except Exception as e:
            print(f"[ERRO ENVIO] {e}")
```

2) *Mapeamento dos serviços*: Esta função atua como um tradutor para facilitar a legibilidade dos dados. Ela utiliza um dicionário interno para correlacionar portas de transporte conhecidas (TCP/UDP) aos seus respectivos serviços de rede. Se o tráfego for capturado em uma porta não especificada, o serviço é automaticamente classificado como *DESCONHECIDO*.

Listing 4. Mapeamento de portas para identificação de serviços (lista completa: SSH, DNS, DHCP, HTTP, HTTPS e uma porta de testes).

```
def get_service(protocol, port):
    if protocol == "ICMP":
        return "ICMP"

    services = {
        22: "SSH", 53: "DNS", 67: "DHCP", 68: "DHCP",
        80: "HTTP", 443: "HTTPS", 8000: "HTTP-TESTE"
    }
    return services.get(port, "DESCONHECIDO")
```

3) *Processamento dos pacotes*: É o núcleo operacional da aplicação, disparado automaticamente pelo Scapy. A função dissection dos cabeçalhos, extrai IPs e portas, e deduz a qual VLAN a comunicação pertence com base no nome da interface lógica. A partir desses dados, ela gera uma chave única e atualiza as métricas daquele fluxo específico, somando pacotes e bytes.

Listing 5. Extração de dados e geração da chave única do fluxo.

```
def process(packet):
    # [...] Verificações de IP ignoradas por brevidade

    interface = packet.sniffed_on
    vlan = interface.split(".")[1] if "." in interface else "N/A"

    # Chave única de identificação da conexão
    flow_key = (interface, src_ip, dst_ip, src_port, dst_port, protocol)
    now = datetime.now()

    # Cria o fluxo se ainda não existir, com marca de tempo inicial
    if flow_key not in flows:
        flows[flow_key] = {"packets": 0, "bytes": 0, "first_seen": now, "last_seen": now}

    # Atualiza métricas do fluxo (novo ou já existente)
    flows[flow_key]["packets"] += 1
    flows[flow_key]["bytes"] += len(packet)
    flows[flow_key]["last_seen"] = now
```

4) *Exportação dos fluxos e Estatísticas*: Além de alimentar a API, esta rotina serve como mecanismo local de consolidação. Ela itera sobre os fluxos na memória para calcular métricas avançadas, como a duração total da comunicação e a taxa de transferência média em bytes por segundo, salvando tudo em um arquivo JSON.

Listing 6. Cálculo de métricas de rede para exportação.

```
def export_json():
    # [...] Estruturação da lista de dados

    duration = (stats["last_seen"] - stats["first_seen"]).total_seconds()
    rate = stats["bytes"] / duration if duration > 0 else 0
    service = get_service(protocol, dst_port)

    # [...] Exportação para flows.json omitida por brevidade
```

5) *Remoção de fluxos expirados*: Implementada para assegurar o gerenciamento eficiente dos recursos da VM. A função examina todas as conexões armazenadas e compara a data do último tráfego recebido com a hora atual. Se a inatividade superar o limite estipulado pelo `FLOW_TIMEOUT`, o fluxo é classificado como expirado e é removido da memória.

Listing 7. Lógica de expiração de fluxos inativos.

```
def remove_expired_flows():
    now = datetime.now()
    for flow in list(flows.keys()):
        idle_time = (now - flows[flow]["last_seen"]).total_seconds()

        if idle_time > FLOW_TIMEOUT:
            del flows[flow] # Libera a memória RAM
```

6) *Gerenciamento dos fluxos em Segundo Plano*: Para que o sistema consiga capturar pacotes sem interrupções, as rotinas de gerenciamento operam em uma segunda thread. A cada intervalo estipulado, ela orquestra a limpeza de memória, dispara a exportação, registra um resumo de status no terminal e empacota novos fluxos para a API.

Listing 8. Thread de orquestração rodando em loop contínuo.

```
def print_flow_table():
    while True:
        time.sleep(10)

        if not flows_changed:
            continue

        remove_expired_flows()
        export_json()
        # [...] Lógica de empacotamento (batch) e envio para send_flow()
```

7) *Justificativa da Arquitetura de Concorrência*: A escolha por uma arquitetura baseada em duas threads decorre diretamente da natureza bloqueante da função de captura do Scapy. A chamada `sniff()`, responsável por escutar as interfaces de rede, permanece ocupada indefinidamente processando cada pacote recebido através da função de callback `process()`. Caso essa fosse a única linha de execução do programa, qualquer tarefa periódica, como a remoção de fluxos expirados, a exportação para o arquivo JSON ou o envio de lotes para a API da VM1, precisaria ser executada entre a chegada de um pacote e outro, o que tornaria essas rotinas dependentes do

volume de tráfego instantâneo e poderia atrasar ou até bloquear a captura em momentos de rajada de pacotes.

Para resolver esse problema, a aplicação foi dividida em duas linhas de execução independentes. A thread principal permanece dedicada exclusivamente à captura, sendo responsável apenas por atualizar o dicionário `flows` a cada pacote recebido. Uma segunda thread, criada como *daemon* através da função `print_flow_table()`, executa em um laço próprio com intervalo fixo de dez segundos, cuidando de todas as tarefas de gerenciamento: limpeza de fluxos inativos, exportação do arquivo local e comunicação com a API REST. Dessa forma, a velocidade de captura nunca fica condicionada à velocidade de processamento ou de envio dos dados.

Como o gargalo do sistema é predominantemente de entrada e saída (leitura de pacotes na interface de rede e requisições HTTP), e não de processamento intensivo de CPU, o uso de threads em Python se mostrou adequado apesar do *Global Interpreter Lock* (GIL). O GIL impede a execução simultânea de bytecode Python em múltiplos núcleos, mas é liberado automaticamente durante operações de I/O, como chamadas de rede e leitura de sockets. Isso permite que a thread de captura e a thread de gerenciamento avancem de forma efetivamente concorrente na maior parte do tempo, mesmo compartilhando um único núcleo de processamento, compatível com o perfil de hardware reduzido da VM4 (1 vCPU).

Por se tratar de duas threads acessando a mesma estrutura de dados (`flows`), o compartilhamento de estado exigiu atenção. Optou-se por manter as operações de escrita sobre o dicionário simples e atômicas dentro de cada thread, apoiando-se no fato de que operações básicas de inserção e atualização de chaves em dicionários Python são protegidas pelo próprio GIL. A variável booleana `flows_changed` foi usada como sinalizador leve entre as threads, evitando que a thread de gerenciamento execute processamento desnecessário quando nenhum pacote novo chegou. Reconhece-se, entretanto, que uma versão futura da aplicação se beneficiaria do uso explícito de um objeto `threading.Lock` ao redor das seções críticas, tornando o controle de concorrência independente de detalhes de implementação do interpretador.

8) *Inicialização do Flow Generator*: O ponto de ignição do script. Aqui são declaradas as dez subinterfaces virtuais monitoradas (VLANs 10, 20, 30, 50 e 60, replicadas nos dois troncos `ens3` e `ens7`), a thread de gerenciamento é acionada e o módulo de captura do Scapy é iniciado com o parâmetro `store=False`, crucial para evitar vazamento de memória durante análises prolongadas.

Listing 9. Disparo da aplicação e da captura em modo não-persistente.

```
if __name__ == "__main__":
    interfaces_to_monitor = [
        "ens3.10", "ens3.20", "ens3.30",
        "ens3.50", "ens3.60",
        "ens7.10", "ens7.20", "ens7.30",
        "ens7.50", "ens7.60",
    ]

    table_thread = threading.Thread(target=
    print_flow_table, daemon=True)
    table_thread.start()

    sniff(iface=interfaces_to_monitor, prn=
    process, store=False)
```

D. Arquitetura da VM4

Esta seção apresenta a arquitetura lógica da solução implementada na VM4, descrevendo como a máquina foi integrada à infraestrutura distribuída do projeto e como o Flow Generator processa os pacotes recebidos até sua transmissão para o servidor central.

A arquitetura da Máquina Virtual 4 (VM4) funciona através do recebimento e processamento contínuo dos dados capturados na rede. O caminho lógico desse tráfego na infraestrutura e a arquitetura interna e etapas de processamento da aplicação Flow Generator, está ilustrado no diagrama de blocos da Figura 5 e na Figura 6

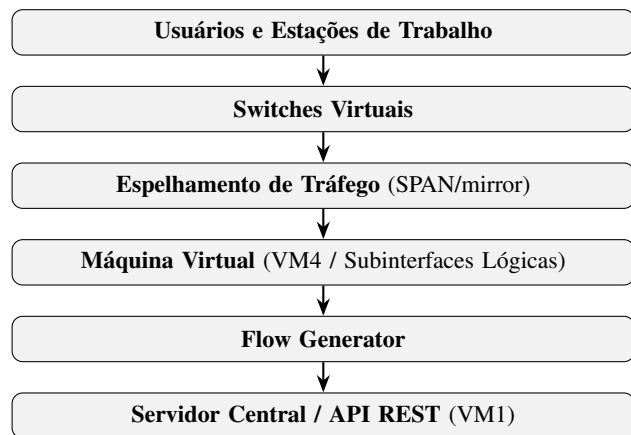


Figura 5. Caminho lógico e esteira de processamento do fluxo de dados da infraestrutura distribuída.

Para conseguir enxergar todo o tráfego das redes, a VM4 foi colocada na rede de monitoramento (VLAN 40). A máquina foi conectada usando duas interfaces de rede funcionando como troncos (*Trunks*), o que permite passar o tráfego de várias VLANs pelo mesmo cabo virtual: a *ens3* ligada ao switch principal (VM_SW1) e a *ens7* ligada ao switch secundário (VM_SW2).

No entanto, para facilitar a separação dos pacotes de cada rede, a VM4 não usa apenas essas duas interfaces principais. Por cima delas, foram criadas dez subinterfaces virtuais (uma para cada VLAN de produção, sendo elas: 10, 20, 30, 50

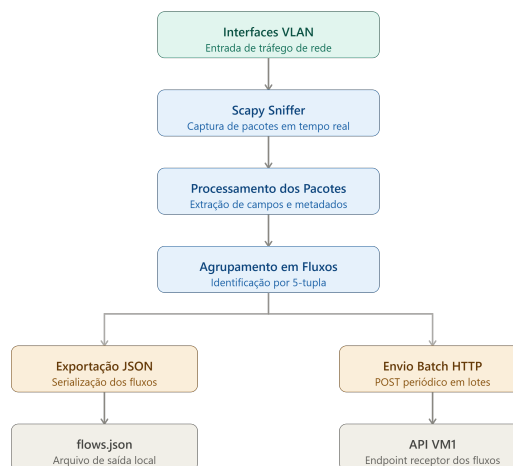


Figura 6. Arquitetura interna e etapas de processamento da aplicação Flow Generator.

e 60, em cada tronco). A VLAN 40, por corresponder ao próprio segmento de monitoramento/gerência, não é replicada em subinterfaces de captura. Isso permite que a aplicação saiba rapidamente de qual rede o tráfego veio.

E. Configuração Realizada e Estrutura de Interfaces

1) Configuração Realizada e Estrutura de Interfaces:

É importante ressaltar, de forma preliminar, que toda a infraestrutura e configuração lógica das interfaces de rede conectadas à VM4 foi projetada e implementada pelo Aluno 5 (módulo VM5), por meio do script automatizado `configurar-captura-vlans.sh`. A partir dessa base de observabilidade, a transição da VM4 para o ambiente de produção real exigiu ajustes complementares no sistema operacional focados na recepção passiva dos fluxos e no roteamento, mantendo o alinhamento com a arquitetura geral projetada pelas demais equipes.

A primeira configuração crítica consistiu no direcionamento do tráfego da sonda. Conforme a topologia adaptada e as orientações fornecidas em conjunto pelos alunos 5 e 11, a rota *default* (gateway padrão) das interfaces de gerência da VM4 foi configurada apontando estritamente para o firewall (VM11), garantindo a integração ao roteamento seguro inter-VLAN.

Para a recepção exclusiva do tráfego espelhado vindo dos dois switches virtuais corporativos (VM_SW1 e VM_SW2), foram isoladas apenas as interfaces físicas de captura da máquina virtual, omitindo-se qualquer interface de gerência local. O mapeamento dessas interfaces físicas está detalhado na listagem abaixo.

VLANs configuradas

O script cria subinterfaces para as seguintes VLANs: 10, 20, 30, 50 e 60. Essas VLANs representam os principais segmentos de produção monitorados pela VM4.

A VLAN 40 não foi incluída nesse script porque está relacionada ao segmento de monitoramento/gerência e ao próprio ambiente de captura.

Subinterfaces criadas

Para a interface `ens3`, são criadas:

```
ens3.10
ens3.20
ens3.30
ens3.50
ens3.60
```

Para a interface `ens7`, são criadas:

```
ens7.10
ens7.20
ens7.30
ens7.50
ens7.60
```

F. Containerização com Docker

Concluído o desenvolvimento e a validação funcional do *Flow Generator*, partiu-se para a etapa de containerização da aplicação, com o objetivo de padronizar o ambiente de execução e facilitar a reprodução do sistema em outras máquinas da infraestrutura.

1) *Definição das dependências*: O primeiro passo consistiu na criação do arquivo `requirements.txt`, contendo as duas bibliotecas Python externas utilizadas pela aplicação.

Listing 10. Dependências Python da aplicação.

```
scapy
requests
```

2) *Construção da imagem*: Em seguida, foi escrito o `Dockerfile`, responsável por definir a imagem da aplicação. Optou-se pela imagem base `python:3.12-slim`, por oferecer um ambiente Python funcional com tamanho reduzido, adequado ao perfil de hardware limitado (1 vCPU, 1 GB de RAM) atribuído à VM4.

Listing 11. `Dockerfile` utilizado na construção da imagem do Flow Generator.

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "flow_generator.py"]
```

A imagem foi então construída localmente com o comando:

Listing 12. Construção da imagem Docker.

```
docker build -t flow-generator .
```

Sempre que o arquivo `flow_generator.py` era alterado durante o desenvolvimento, a imagem precisava ser reconstruída com o mesmo comando, para que o container passasse a refletir a versão mais recente do código.

3) *Execução do container*: A execução do container exigiu duas flags específicas. A opção `-net=host` foi necessária para que o container enxergasse diretamente as subinterfaces VLAN criadas no sistema operacional da VM4 (`ens3.X` e `ens7.X`), já que, em modo de rede isolado (*bridge*), o Docker não teria acesso a essas interfaces. A opção `-privileged` foi necessária para permitir que a biblioteca Scapy, dentro do container, abrisse *sockets* em modo promíscuo e capturasse o tráfego espelhado.

Listing 13. Execução inicial do container.

```
docker run --net=host --privileged flow-generator
```

4) *Ajuste de limites de log*: Durante a operação contínua do container, observou-se, por meio do comando `df -h`, que a partição responsável por armazenar os dados do Docker (`/var/lib/docker`) apresentava crescimento constante, aproximando-se da capacidade total do disco da VM. A investigação revelou que a causa não estava no arquivo `flows.json`, sobrescrito a cada ciclo e, portanto, de tamanho limitado mas sim no mecanismo padrão de *logging* do Docker (*driver json-file*), que grava todo o conteúdo impresso pela aplicação em um arquivo de log sem limite de tamanho por padrão.

Duas medidas foram adotadas para corrigir o problema. Primeiro, a rotina de exibição em terminal foi simplificada: em vez de imprimir a tabela completa de fluxos a cada ciclo de 10 segundos, a aplicação passou a registrar apenas uma linha de resumo por ciclo, reduzindo drasticamente o volume de dados gravados em log. Segundo, o container passou a ser executado com limites explícitos de rotação de log, garantindo uma proteção adicional independentemente do volume de saída da aplicação.

Listing 14. Execução final do container, com nome fixo e limites de log configurados.

```
docker run -d \
  --name flow-generator \
  --net=host \
  --privileged \
  --log-opt max-size=20m \
  --log-opt max-file=3 \
  flow-generator
```

Com essa configuração, o Docker mantém no máximo três arquivos de log de 20 MB cada por container (60 MB no total), descartando automaticamente o conteúdo mais antigo quando o limite é atingido, e eliminando o risco de esgotamento do disco da VM durante execuções prolongadas.

5) *Verificação e monitoramento*: Após a execução, a existência e o tamanho da imagem foram conferidos com `docker images`, o estado do container com `docker ps`, e o consumo agregado de disco pelo ecossistema Doc-

ker com docker system df. O acompanhamento dos logs em tempo real foi feito com `docker logs -f flow-generator`, permitindo observar o resumo de status impresso a cada ciclo e confirmar visualmente o funcionamento contínuo da aplicação.

G. Testes Realizados

O módulo *Flow Generator* foi submetido a uma série de testes durante o seu desenvolvimento. O objetivo principal foi validar todas as funcionalidades implementadas, garantindo o correto funcionamento do sistema desde a captura dos pacotes até a comunicação com a API REST.

1) *Validação da Captura de Pacotes*: Inicialmente, foi verificado se a VM4 estava recebendo corretamente o tráfego de rede espelhado pelos switches virtuais. Para isso, utilizaram-se comandos do próprio sistema operacional Linux, como o `tcpdump`, permitindo confirmar a chegada dos pacotes nas subinterfaces VLAN.

Listing 15. Comando para validação da captura de tráfego na VLAN 10.

```
tcpdump -i ens3.10
```

Após essa validação, a aplicação *Flow Generator*, baseada na biblioteca Scapy, foi executada. Os resultados confirmaram que todos os pacotes capturados eram devidamente encaminhados para a função principal de processamento, denominada `process()`.

2) *Teste de Protocolos e Serviços*: Para verificar a capacidade de identificação da aplicação, foram gerados diferentes tipos de tráfego a partir da máquina virtual de testes (VMTESTE):

- ICMP (*ping*);
- HTTP, utilizando um servidor Python;
- SSH

Observou-se que a aplicação identificou corretamente o endereço IP de origem, o endereço IP de destino, as portas envolvidas, o protocolo de transporte e o serviço correspondente, de acordo com o mapeamento configurado.

3) *Teste da Geração de Fluxos*: Foi realizada uma sequência contínua de comunicações entre as máquinas virtuais para certificar que múltiplos pacotes pertencentes a uma mesma conexão eram agrupados em um único fluxo. Durante o teste, foram analisadas as seguintes métricas:

- Quantidade de pacotes;
- Quantidade de bytes;
- Duração do fluxo;
- Taxa média de transmissão.

Os resultados registrados pelo sistema foram compatíveis com o volume de tráfego efetivamente gerado.

4) *Teste da Exportação JSON*: Após o processamento dos fluxos, verificou-se o conteúdo do arquivo local `flows.json`. Foi confirmada a presença e a exatidão dos seguintes campos: interface, VLAN, IP de origem, IP de destino, protocolo, serviço, número de pacotes, quantidade de bytes, duração, bytes por segundo, além do horário inicial e

final da conexão. O arquivo foi atualizado corretamente a cada ciclo de execução.

5) *Teste da Comunicação com a API REST*: Este teste validou o envio automático dos dados estruturados para a API hospedada na VM1. Durante a execução, verificou-se:

- O estabelecimento da conexão HTTP;
- O envio do lote de informações em formato JSON;
- O recebimento do código de sucesso HTTP 200/201;
- O tratamento adequado de erros nos momentos em que a API estava indisponível.

Além disso, comprovou-se que um mesmo fluxo não era enviado repetidamente, confirmando a eficácia do mecanismo de controle implementado pela variável `sent_flows`.

6) *Teste do Mecanismo de Timeout*: Para assegurar a remoção automática de fluxos antigos e evitar o consumo excessivo de memória, a geração de tráfego foi completamente interrompida.

Listing 16. Tempo limite configurado para a expiração de fluxos.

```
FLOW_TIMEOUT = 60 # Tempo em segundos
```

Após o intervalo de 60 segundos, definido pela variável `FLOW_TIMEOUT`, os fluxos inativos foram removidos da memória do sistema de forma automática, liberando recursos e mantendo a estabilidade da aplicação.

7) *Teste da Execução Concorrente*: Por fim, foi avaliado o funcionamento simultâneo das duas *threads* da aplicação. Durante a execução contínua, observou-se que:

- A captura de pacotes não foi interrompida;
- A exportação dos dados e o envio para a API ocorreram de forma paralela;
- Não houve travamentos ou falhas na sincronização das tarefas.

Dessa forma, a arquitetura adotada mostrou-se adequada e confiável para operações de monitoramento contínuo.

IV. RESULTADOS

Após a conclusão do desenvolvimento, da containerização e da bateria de testes descrita na Seção anterior, a aplicação *Flow Generator* foi colocada em execução contínua na VM4, permitindo avaliar seu comportamento em condições próximas às de operação real.

A. Geração e consolidação dos fluxos

Ao longo da execução, a aplicação demonstrou capacidade de identificar corretamente o tráfego proveniente das cinco VLANs monitoradas (10, 20, 30, 50 e 60), agrupando os pacotes observados em fluxos consistentes com base na tupla (interface, IP de origem, IP de destino, porta de origem, porta de destino, protocolo). Fluxos de protocolos TCP, UDP e ICMP foram corretamente diferenciados, e o serviço de aplicação associado a cada fluxo (SSH, DNS, DHCP, HTTP, HTTPS ou desconhecido) foi identificado de acordo com a porta de destino observada.

B. Envio de resumo e estatísticas para a VM1

O principal resultado observado na integração com o servidor central foi a confirmação de que os fluxos consolidados eram efetivamente transmitidos, em lotes (*batches*), para a API REST hospedada na VM1, no endpoint `POST /api/v1/flows/batch`. A cada ciclo de 10 segundos em que novos pacotes eram capturados, a aplicação executava a seguinte sequência: remoção dos fluxos inativos há mais de 60 segundos, atualização do arquivo local `flows.json`, montagem do lote com os fluxos ainda não enviados anteriormente (controlado pelo conjunto `sent_flows`, evitando duplicidade) e envio do lote via requisição HTTP POST.

O sucesso de cada envio era confirmado pela aplicação a partir do código de status HTTP retornado pela VM1 (200 ou 201), e registrado de forma resumida no terminal, conforme ilustrado na Figura 7.

```
VM1 OK] 2 fluxos enviados.
[23:54:52] fluxos ativos: 2 | novos enviados: 2 | pacotes totais: 3 | bytes totais: 1005
VM1 OK] 6 fluxos enviados.
[23:55:52] fluxos ativos: 8 | novos enviados: 6 | pacotes totais: 43 | bytes totais: 11894
[23:57:02] fluxos ativos: 2 | novos enviados: 0 | pacotes totais: 46 | bytes totais: 12899
[23:58:02] fluxos ativos: 2 | novos enviados: 0 | pacotes totais: 49 | bytes totais: 13904
[23:59:02] fluxos ativos: 2 | novos enviados: 0 | pacotes totais: 52 | bytes totais: 14909
```

Figura 7. Confirmação de envio de um lote de fluxos para a VM1.

Além da confirmação pontual de cada lote, a aplicação passou a registrar, ao final de cada ciclo, um resumo consolidado do estado da captura – incluindo o número de fluxos ativos em memória, a quantidade de fluxos inéditos enviados naquele ciclo e os totais acumulados de pacotes e bytes observados desde o início da execução, conforme a Figura 8. Esse resumo substituiu a impressão da tabela completa de fluxos discutida na Seção III-F4, preservando a visibilidade operacional do sistema sem comprometer o armazenamento em disco da VM ao longo de execuções prolongadas.

```
root@ubuntu:~/aluno4# docker run --net-host --privileged flow-generator
=====
Flow Generator iniciado
=====
Interfaces monitoradas:
- ens3.10
- ens3.20
- ens3.30
- ens3.50
- ens3.60
- ens7.10
- ens7.20
- ens7.30
- ens7.50
- ens7.60
[VM1 OK] 2 fluxos enviados.
[23:54:52] fluxos ativos: 2 | novos enviados: 2 | pacotes totais: 3 | bytes totais: 1005
[VM1 OK] 6 fluxos enviados.
[23:55:52] fluxos ativos: 8 | novos enviados: 6 | pacotes totais: 43 | bytes totais: 11894
[23:57:02] fluxos ativos: 2 | novos enviados: 0 | pacotes totais: 46 | bytes totais: 12899
[23:58:02] fluxos ativos: 2 | novos enviados: 0 | pacotes totais: 49 | bytes totais: 13904
[23:59:02] fluxos ativos: 2 | novos enviados: 0 | pacotes totais: 52 | bytes totais: 14909
[00:00:12] fluxos ativos: 2 | novos enviados: 0 | pacotes totais: 55 | bytes totais: 15914
[00:01:12] fluxos ativos: 2 | novos enviados: 0 | pacotes totais: 58 | bytes totais: 16919
```

Figura 8. Resumo de status impresso ao final de cada ciclo.

A consistência dos dados recebidos pela VM1 foi verificada externamente por meio de uma requisição url de resumo do servidor central, executada no browser:

Listing 17. Verificação do resumo de fluxos recebidos, executada na VM1.

```
http://10.10.1.2/docs
```

Os valores retornados por esse endpoint mostraram-se coerentes com os totais de pacotes e bytes registrados localmente

pela VM4, confirmando a integridade da transmissão entre os dois módulos e validando o funcionamento fim a fim da integração da VM4 com o servidor central da plataforma. A Figura 9

```
{
  "interface": "ens7.10",
  "vlan": "10",
  "src_ip": "0.0.0.0",
  "dst_ip": "255.255.255.255",
  "src_port": 68,
  "dst_port": 67,
  "protocol": "UDP",
  "service": "DHCP",
  "packets": 3,
  "bytes": 996,
  "duration": 127.49,
  "bytes_per_second": 7.81,
  "first_seen": "2026-06-30 23:04:42",
  "last_seen": "2026-06-30 23:06:50"
},
{
  "interface": "ens3.10",
  "vlan": "10",
  "src_ip": "0.0.0.0",
  "dst_ip": "255.255.255.255",
  "src_port": 68,
  "dst_port": 67,
  "protocol": "UDP",
  "service": "DHCP",
  "packets": 6,
  "bytes": 1992,
  "duration": 127.49,
  "bytes_per_second": 15.62,
  "first_seen": "2026-06-30 23:04:42",
  "last_seen": "2026-06-30 23:06:50"
},
{
  "interface": "ens3.40",
  "vlan": "40",
  "src_ip": "10.0.40.30",
  "dst_ip": "149.154.166.110",
  "src_port": 44274,
  "dst_port": 443,
  "protocol": "TCP",
  "service": "HTTPS",
  "packets": 12,
  "bytes": 2837,
  "duration": 0.98,
  "bytes_per_second": 2889.56,
  "first_seen": "2026-06-30 23:06:46",
  "last_seen": "2026-06-30 23:06:47"
},
}
```

Figura 9. Fluxos enviados para a VM1 com todas as informações

V. USO DE INTELIGÊNCIA ARTIFICIAL

Durante o ciclo de desenvolvimento, ferramentas de Inteligência Artificial Gerativa foram utilizadas como mecanismos de suporte técnico. As consultas abrangeram tópicos pontuais, tais como: esclarecimento teórico sobre restrições de *Port Security* em redes SDN; auxílio na depuração (*debugging*) de códigos de erro envolvendo a alteração de tamanho de dicionários em concorrência no Python; e orientação na estruturação sintática do arquivo `Dockerfile`. Ressalta-se que o *design* lógico, a codificação, a configuração do *kernel*, a execução das rotinas de teste e todas as decisões técnicas finais foram elaboradas e validadas exclusivamente pelo autor do projeto.

VI. CONCLUSÃO

O desenvolvimento do módulo VM4 permitiu consolidar, na prática, conceitos de captura passiva de tráfego, programação concorrente e integração entre serviços distribuídos em um ambiente de nuvem privada. A escolha da biblioteca Scapy se mostrou adequada ao contexto do projeto, pois eliminou a dependência de processos externos de captura e possibilitou o controle total sobre a extração dos campos necessários para a

construção dos fluxos, algo que ferramentas como o TShark ofereceriam com maior custo de integração.

A separação da aplicação em duas threads, uma dedicada à captura contínua e outra à orquestração periódica de limpeza, exportação e envio de dados, mostrou ser a solução mais adequada ao perfil de hardware limitado da VM4, garantindo que o processamento de gerenciamento nunca comprometesse a recepção do tráfego espelhado. Da mesma forma, a containerização via Docker, com o uso das flags `-net=host` e `-privileged`, permitiu que a aplicação acessasse diretamente as subinterfaces VLAN do sistema operacional, sem abrir mão da portabilidade proporcionada pelo container.

Os testes realizados confirmaram o correto funcionamento de todas as etapas do pipeline, desde a identificação de VLANs e protocolos até o envio consolidado dos fluxos para a API central hospedada na VM1. O principal desafio encontrado ao longo do desenvolvimento foi o crescimento descontrolado dos logs gerados pelo driver padrão do Docker, resolvido por meio da simplificação da rotina de exibição em terminal e da configuração explícita de limites de rotação de log. Esse ajuste, somado às demais decisões de projeto, resultou em um sistema estável, capaz de operar de forma contínua e de se integrar de maneira consistente ao restante da plataforma distribuída desenvolvida pelo grupo.

Como trabalhos futuros, algumas melhorias podem ser incorporadas ao módulo para ampliar sua robustez e escalabilidade. Do ponto de vista da concorrência, a substituição do controle implícito baseado no GIL por um objeto `threading.Lock` explícito ao redor das seções críticas tornaria o compartilhamento do dicionário `flows` mais seguro e independente de detalhes de implementação do interpretador Python. Outra possibilidade é a adoção de uma fila (`queue.Queue`) entre a thread de captura e a thread de gerenciamento, desacoplando ainda mais as duas responsabilidades e reduzindo o risco de contenção sobre a mesma estrutura de dados.

Em relação à captura, a aplicação poderia evoluir para persistir os fluxos ativos em um mecanismo de armazenamento mais resiliente do que a memória RAM, como um banco de dados local leve (por exemplo, SQLite), evitando a perda de dados em caso de reinicialização do container antes do envio para a VM1. A ampliação do dicionário de serviços conhecidos, hoje limitado a poucas portas, também tornaria a identificação de protocolos mais completa em cenários com maior diversidade de tráfego.

No que diz respeito à confiabilidade da comunicação com o servidor central, seria interessante implementar um mecanismo de reenvio automático (*retry*) com backoff exponencial para os lotes que falharem no envio, além de um buffer local temporário para armazenar fluxos não confirmados durante períodos de indisponibilidade da API da VM1. Por fim, a exposição de métricas internas da própria aplicação, como taxa de captura, tamanho do dicionário de fluxos ativos e latência de envio, permitiria observar o funcionamento do Flow Generator com o mesmo nível de detalhe que ele hoje oferece para o restante da rede monitorada.